数据与算法

CH14 复习提纲

北京科技大学
计算机与通信工程学院
杜　冰/皇甫伟

课程内容纲要

绪论	数值计算
数据与算法概念	数值计算基本概念
数学模型	线性方程组
算法复杂度	非线性方程
非数值型数据与算法	拟合与插值（优化）
二元关系与ADT	随机算法
线性表	**各章算法思想**
栈与队列	
树与二叉树	
图	
查找与排序	

绪论

- 数据 vs 算法
  - 数据是信息的载体，算法是处理数据的系统
- 算法的定义
  - 有限步骤，基本运算，无歧义，输入，输出
- 算法的评价
  - 正确，高效，健壮，优雅
- 算法分析
  - 时间复杂度，空间复杂度
  - 函数渐进分析，大O符号，算法分析的方法
- 数学模型

二元关系与ADT

- 数据的逻辑与存储结构
  - 线性结构(顺序，链式)，非线性结构
- 抽象数据类型(ADT)

线性表

- 线性表
  - 相同性质的数据元素顺序排列形成的有限序列
- 顺序表
  - 数据的逻辑与物理结构均为顺序排列
  - 顺序表的基本操作
- 链表
  - 数据的物理结构不一定为顺序排列
  - 用附加的指针存储逻辑顺序关系
  - 单链表、双向链表的基本操作

栈与队列

- 栈与队列
  - 简化的线性表：仅操作特定位置的元素
- 栈(stack)
  - 仅操作栈顶位置元素
  - 后进先出：入栈(push)、出栈(pop)
  - 顺序栈：溢出处理、双顺序栈共享
  - 链式栈：基本操作与特性
- 栈的应用
  - 适用于将问题暂存转而解决子问题
  - 表达式求值，后缀表达式，递归

栈与队列

- 递归(recursion)
  - 一个过程直接或者间接地调用自己
  - 递归程序的编写：思路清晰，代码简洁
  - 递归程序的问题：效率低下，递归调用栈消耗空间
  - 递归的消去：递归与循环控制结构的表达能力等价
  - 尾递归消去，模拟递归调用栈
- 队列(list)
  - 仅操作队头、队尾的数据元素
  - 先进先出：入队、出队
  - 顺序队列，循环队列：元素个数，队空、队满
  - 链式队列

尾递归、递归消除

树与二叉树

- 树(Tree)
  - 层次的非线性结构
  - 树的递归定义，子树
  - 结点、度、孩子/双亲、路径、深度 ...
- 树的性质
  - 结点数等于结点的度的总和+1
  - 度为k的树，第i层至多有k^{i-1}个结点
  - 度为k且深度为h的树，至多有$(k^h-1)/(k-1)$个结点
  - 具有n个结点的度为k的树的最小深度为$\lceil log_k(n(k-1)+1)\rceil$

树与二叉树

- 二叉树
  - 度不超过2的有序树
- 二叉树的性质
  - 第i层至多有2^{i-1}个结点
  - 深度为h的二叉树至多有$2^h - 1$个结点
  - 任何非空二叉树，叶子结点数为n_0，度为2的结点数为n_2，则一定有：$n_0 = n_2 + 1$
  - 具有n个结点的完全二叉树的深度$h = [log_2(n+1)]$
  - 结点与编号(自上而下，自左而右)

树与二叉树

- 完全二叉树 vs 满二叉树
- 二叉树的实现
  - 顺序表示
  - 链式表示
- 二叉树的遍历及其实现
  - 前/中/后序遍历
  - 层序遍历
- 最优二叉树(Huffman树)
  - 树的带权路径长度
  - Huffman树的构造与最优二进制编码(复杂度？)

树与二叉树

- 二叉搜索树
  - 每个结点用一个关键字唯一标识，关键字可比大小
  - 左子树上所有结点的关键字都小于根结点
  - 右子树上所有结点的关键字都大于根结点
  - 左、右子树都分别是二叉搜索树
- 二叉搜索树的操作
  - 操作性能
  - 查找、插入、删除、构造、旋转（AVL树）

图

- 图的基本概念
  - 顶点、边、邻接顶点
  - 有向图、无向图、简单图、多重图、赋权图、子图
  - 无向完全图、有向完全图、稀疏图
  - 顶点的度、出度、入度、握手定理
  - 路径、简单路径、回路、简单回路
  - 连通、连通图、连通分量、强连通
- 图的存储
  - 邻接矩阵、邻接表
  - 图基本操作的实现

图

- 图的算法
  - 图的遍历：深度优先、广度优先
  - 最小生成树：Prim算法、Kruskal算法
  - 最短路径：Dijkstra算法、Floyd算法

查找与排序

- 查找
  - 关键字、查找表、静态查找结构、动态查找结构
  - 平均查找长度ASL
- 线性查找表
  - 无序序列的顺序查找及实现
  - 有序序列的折半查找及实现:递归、非递归
- 二叉搜索树如何实现查找
- 堆
  - 子节点总是大于根节点：最小堆
  - 插入删除要进行堆化，自底向上建堆

查找与排序

- 散列表
  - 存储位置=H(关键字)，常数实现查找性能
  - 散列函数
  - 散列的性能
- 冲突处理
  - 链地址法（开散列法）
  - 开放定址（闭散列法）：线性探测、双重散列
  - 元素的插入、查找、删除

查找与排序

- 排序
  - 关键字、排序的稳定性、内排序与外排序
  - 性能评估（空间时间复杂度）：比较次数、移动次数、空间消耗
- 基本排序方法
  - 冒泡排序
  - 插入排序：直接、折半、希尔
  - 选择排序
- 快速排序与归并排序
- 计数排序
- 堆排序
- 排序算法的性能分析与比较

性能比较

排序方法	比较次数		移动次数		稳定性	附加存储	
	最好	最差	最好	最差		最好	最差
直接插入	$O(n)$	$O(n^2)$	$O(n)/0$	$O(n^2)$	Y	$O(1)$	
折半插入	$O(n\log_2 n)$		$O(n)$	$O(n^2)$	Y	$O(1)$	
冒泡	$O(n)$	$O(n^2)$	0	$O(n^2)$	Y	$O(1)$	
选择	$O(n^2)$		$O(n)$		N	$O(1)$	
快速	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(n^2)$	N	$O(\log_2 n)$	$O(n)$
归并	$O(n\log_2 n)$ 引入了一个与初始序列长度相同的数组来存储合并后的结果，因而不涉及交换				Y	$O(n)$	
堆排序	$O(n\log_2 n)$		$O(n\log_2 n)$		N	$O(1)$	
计数排序	$O(n+k)$，n为输入元素个数，k为待排序列中最大的数				N	$O(k)$	

直接插入排序在程序实现上，通常会用一个临时变量(如temp）保存待插入元素，最后又将temp移动到相应位置上，因而在很多教材上插入会多2次移动操作

数值计算基本概念

- 数值计算特点
  - 变量连续、解空间无限
  - 求满足精确度要求的解
  - 算法的性能：收敛性、稳定性以及解的精确度
- 数值问题特性
  - 适定、不适定、良态、病态
- 误差分析
  - 精确度，有效数字
  - 绝对误差、相对误差、数据误差
  - 计算误差：截断误差、舍入误差
  - 向前误差、向后误差、条件数

数值计算基本概念

- 浮点数
  - 浮点数的表示：基数、尾数、指数、精度
  - IEEE浮点数系统、正规化
  - 上溢限、下溢限、机器精度
  - 浮点数运算：上溢、下溢、大数吃小数、抵消

线性方程组

- 向量与矩阵
  - 向量的范数、P范数
  - 矩阵：正交、秩、行列式、特征值、正定
  - 雅可比矩阵、海森矩阵
  - 矩阵的范数： $\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$
  - 1-范数： $\|A\|_1 = \max_j \sum_{i=1}^{n} |a_{ij}|$
  - ∞-范数： $\|A\|_\infty = \max_i \sum_{j=1}^{n} |a_{ij}|$
  - 矩阵的条件数： $cond(A) = \|A\| \|A^{-1}\|$
  - 矩阵条件数、线性方程组解的敏感性、残差

线性方程组

- 线性方程组直接解法
  - 解的存在性与唯一性
  - 高斯消去法：实现、复杂度、选主元
  - LU分解，初等消去阵
- 线性方程组迭代解法
  - 不动点迭代形式
  - 雅可比迭代： $A = D + (L + U)$
  - 高斯-赛德尔迭代： $A = (D + L) + U$

非线性方程

- 非线性方程特性
  - 解的存在性、唯一性
  - 介值定理
  - 简单根与重根
  - 求根问题绝对条件数： $1/|f'(x)|$
  - 收敛速度： $\lim_{k\to\infty}\frac{\|e_{k+1}\|}{\|e_k\|^r}=C$
  - 迭代判停
- 非线性方程求解
  - 二分法、试位法：知道根区间，但求解过程简单，收敛性能保证

非线性方程

- 非线性方程求解
  - 不动点迭代：$x_{k+1} = g(x_k)$
    - 若$|g'(x^*)| > 1$，则迭代过程发散；
    - 若$0 < |g'(x^*)| < 1$，则迭代过程线性收敛；
    - 若$|g'(x^*)| = 0$，则迭代过程至少是平方收敛的
  - 牛顿法：
    - 根的附近平方收敛，但初值敏感 $$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$
  - 割线法与反插法

拟合与插值

- 插值
  - 要求函数精确通过数据点
  - 插值问题的解存在性及唯一性
  - 敏感性取决于系数矩阵A的条件数，而A取决于基函数的选取
  - 插值求解方法
    - 多项式插值
    - 拉格朗日插值
    - 牛顿插值

拟合与插值

- 拟合
  - 超定方程组、最小二乘
  - 线性拟合$AX \cong b$，解的存在与唯一，拟合问题敏感性不仅仅决定于$cond(A)$，也取决于b
  - 正规方程法，解的敏感性
  - 机器学习基本概念：过拟合 欠拟合
  - 梯度下降

随机算法

- 元启发算法:
  - 爬山法
  - 模拟退火算法
  - 遗传算法
- Monte Carlo算法
  - 每次得到的解未必正确
  - 多次调用以高概率得到正确解
  - 算法精度

算法思想

- 蛮力法
  - 直接从定义出发，简单但往往效率低
- 分治法
  - 将问题分解成多个同类子问题求解之后再合并
  - 分治法：归并排序
  - 减治法：二分查找法
- 分解
  - LU分解、线性方程组迭代求解

算法思想

- 贪心法
  - 每步都保持当前条件下最优，无法保证全局最优
  - 贪心法：Huffman树、Prim算法、Kruskal算法、Dijkstra算法、最速下降法
- 动态规划
  - 将问题分解为不独立的子问题(阶段)求解
  - 最优化原理：最优策略只能由最优子策略组成
  - 无后效性：当前状态就是历史完整的总结
  - Floyd算法、背包问题
  - 关键在于子问题的设计

算法思想

- 迭代-不动点
  - 非线性方程求解
- 随机仿真：
  - 概率意义的“枚举（穷举）”思想

算法优化策略

- 算法优化策略
  - 提高算法性能的思维方式，具有孤立性和经验性
  - 一些常见的优化方式
    - 使用算设计技术：分治、动态规划
    - 启发式：回溯、分支界限
    - 改变数据结构：二叉搜索树
    - 维护索引结构：有序表、二叉树、散列表
    - 充分利用空间：线索二叉树
    - 时空互换（空间换时间）：散列
    - 算法组合：快速排序+插入排序

图搜索

- 状态空间图构造
- 组合优化问题往往转化为图的搜索
- 蛮力：BFS、DFS
- 回溯法：基于DFS，扩展结点前分析是否需要扩展
- 分支界限法：基于BFS，选择‘最有希望’的结点优先扩展

分数比例

- 期末考试 (50%)
- 实验：3次
  - 线性表+栈、树、图
- 作业：7次
  - 线性表、栈与队列、树、图、查找与排序、数值计算、机器学习+随机

45%

- 考勤 (5%)

期末考试

- 闭卷考试(可使用计算器)
- 考试时间：2024.6.8 上午8:30~10:30
- 考试地点(按照学号顺序到指定考场)：
  - ******
- 试题种类：填空题、选择题、简答题、大题（计算）
- 试题难度覆盖：作业 + 例题
- 答疑

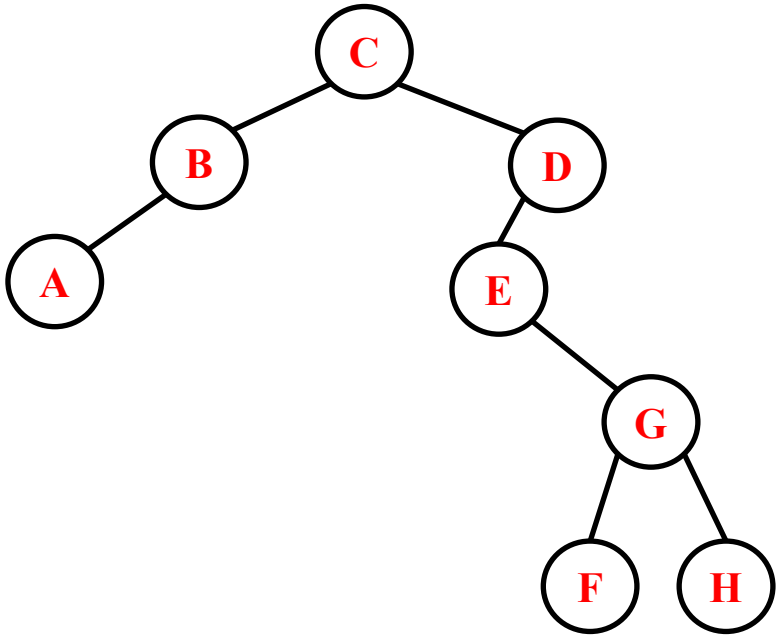
填空题

有若干个算法，其运行时间$T(n)$**和输入规模**n**的关系分别为：**$T_1(n) = 500000n + 4$，$T_2(n) = 0.0001n^2$，$T_3(n) = 64n\log_2 n$，$T_4(n) = 1.01^n$，$T_5(n) = 0.0001n!$，$T_6(n) = n^{1.59}$**，请按照时间复杂度从低到高的顺序进行排列：_____。**

矩阵$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$**，**$A$**的1-范数**$\|A\|_1 =$ _____**，**A**的条件数=**

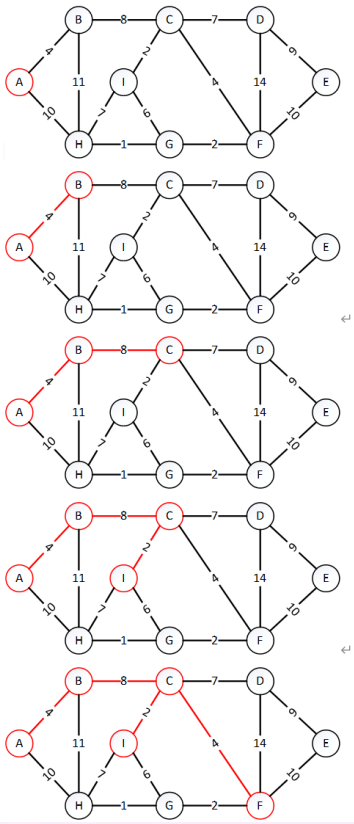
简答题

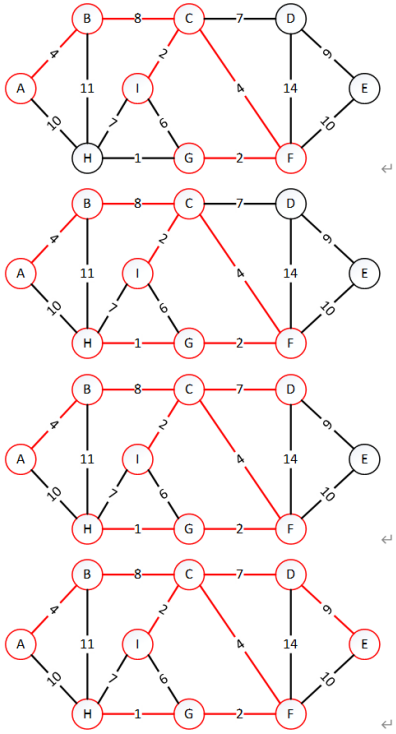
已知一棵二叉树的中序遍历的结果为ABCEFGHD，后序遍历的结果为ABFHGEDC。a）画出这棵二叉树；b）写出这棵二叉树的前序遍历序列。



大题

采用Prim算法求下面无向图的最小生成树，用图示的方法给出算法执行步骤及结果，并分析Prim算法的时间复杂度。（Prim算法是寻找两个集合之间的最短连边加入最小生成树集合，请选取A作为第一个加入的顶点。）





总共 8 条边，每条 1 分

Prim 算法的时间复杂度为 $O(n^2)$　2 分

计算题

- 求矩阵A的LU分解

$$A = \begin{pmatrix} 5 & 2 & -4 \\ 2 & 1 & -2 \\ -4 & -2 & 5 \end{pmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{2}{5} & 1 & 0 \\ -\frac{4}{5} & -2 & 1 \end{bmatrix} \qquad U = \begin{bmatrix} 5 & 2 & -4 \\ 0 & \frac{1}{5} & -\frac{2}{5} \\ 0 & 0 & 1 \end{bmatrix}$$

- 复习提纲里提到的概念、算法是否掌握?
- 作业题是否能独立做出?
- 实验的算法设计思路是否清晰掌握?

/* Q & A */